

PL/SQL Programming

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

Question: Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

1. Create Accounts Table

The screenshot shows the SQL Developer interface. On the left, the 'Navigator' pane displays 'My Schema' with a 'Tables' folder containing 'ACCOUNTS', 'CUSTOMERS', and 'LOANS'. The main editor window, titled '[SQL Worksheet]*', contains the following SQL code:

```
1 CREATE TABLE Accounts (  
2   AccountID NUMBER PRIMARY KEY,  
3   CustomerID NUMBER,  
4   Balance NUMBER  
5 );
```

Below the editor, the 'Script output' tab is active, showing the execution results:

```
SQL> CREATE TABLE Accounts (  
AccountID NUMBER PRIMARY KEY,  
CustomerID NUMBER,  
Balance NUMBER...  
Show more...  
  
Table ACCOUNTS created.  
Elapsed: 00:00:00.021
```

The footer of the application shows '26.5.2 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved'.

2. Insert Sample Data

The screenshot shows the SQL Developer interface. The main editor window contains the following SQL code:

```
1 INSERT INTO Accounts VALUES (101,1,10000);  
2 INSERT INTO Accounts VALUES (102,2,5000);  
3 INSERT INTO Accounts VALUES (103,3,15000);  
4  
5 COMMIT;
```

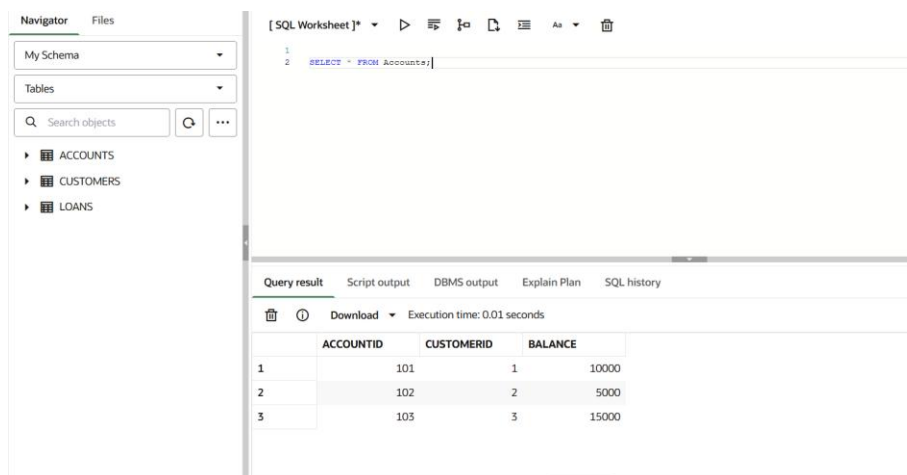
Below the editor, the 'Script output' tab is active, showing the execution results:

```
1 row inserted.  
Elapsed: 00:00:00.002  
  
SQL> COMMIT  
  
Commit complete.  
Elapsed: 00:00:00.002
```

Cognizant Digital Nurture Program

Week 1 Hands-on

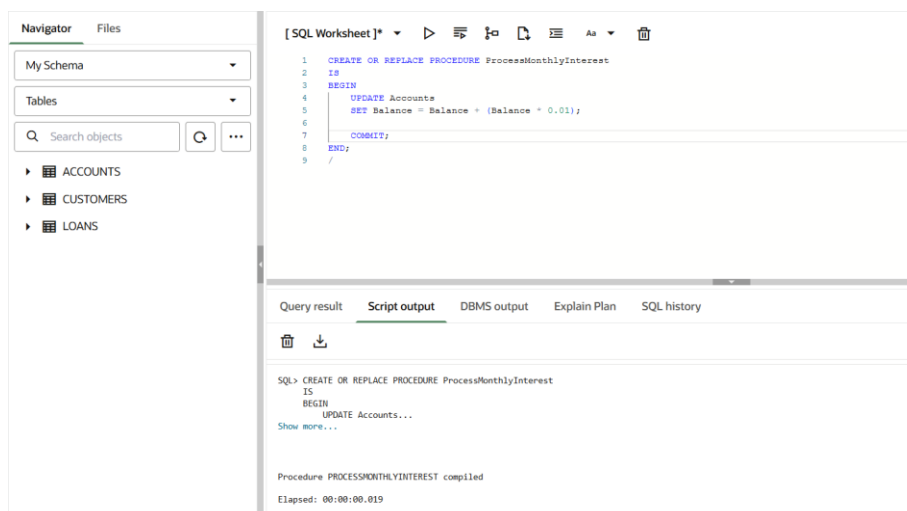
3. Verify Data



The screenshot shows the SQL Developer interface. On the left, the 'Navigator' pane displays 'My Schema' with tables 'ACCOUNTS', 'CUSTOMERS', and 'LOANS'. The main editor shows the query: `SELECT * FROM Accounts;`. The 'Query result' tab is active, displaying a table with 3 rows and 3 columns: ACCOUNTID, CUSTOMERID, and BALANCE.

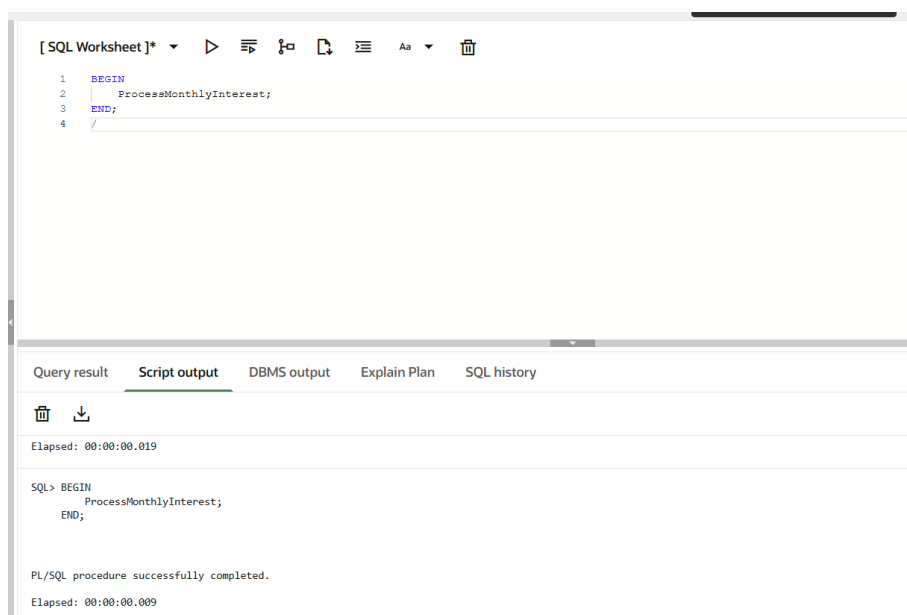
	ACCOUNTID	CUSTOMERID	BALANCE
1	101	1	10000
2	102	2	5000
3	103	3	15000

4. Create Procedure



The screenshot shows the SQL Developer interface. The main editor contains the following SQL code: `CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest IS BEGIN UPDATE Accounts SET Balance = Balance + (Balance * 0.01); COMMIT; END;`. The 'Script output' tab is active, showing the execution of the procedure: `SQL> CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest IS BEGIN UPDATE Accounts...`. Below the script output, it states 'Procedure PROCESSMONTHLYINTEREST compiled' and 'Elapsed: 00:00:00.019'.

5. Execute Procedure



The screenshot shows the SQL Developer interface. The main editor contains the following SQL code: `BEGIN ProcessMonthlyInterest; END;`. The 'Script output' tab is active, showing the execution of the procedure: `SQL> BEGIN ProcessMonthlyInterest; END;`. Below the script output, it states 'PL/SQL procedure successfully completed.' and 'Elapsed: 00:00:00.009'.

Cognizant Digital Nurture Program

Week 1 Hands-on

6. Verify Updated Balance

The screenshot shows the SQL Developer interface. On the left, the Navigator pane displays 'My Schema' with a list of tables: ACCOUNTS, CUSTOMERS, and LOANS. The main workspace shows a SQL worksheet with the query: `SELECT * FROM Accounts;`. Below the query, the 'Query result' tab is active, displaying a table with 3 rows and 4 columns: ACCOUNTID, CUSTOMERID, and BALANCE. The execution time is 0.002 seconds.

	ACCOUNTID	CUSTOMERID	BALANCE
1	101	1	10100
2	102	2	5050
3	103	3	15150

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

Question: Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

1. Create Employees Table

The screenshot shows the SQL Developer interface. On the left, the Navigator pane displays 'My Schema' with a list of tables: ACCOUNTS, CUSTOMERS, EMPLOYEES, and LOANS. The main workspace shows a SQL worksheet with the query: `CREATE TABLE Employees ( EmployeeID NUMBER PRIMARY KEY, EmployeeName VARCHAR2(50), DepartmentID NUMBER, Salary NUMBER );`. Below the query, the 'Script output' tab is active, displaying the SQL command and the message: 'Table EMPLOYEES created.' The elapsed time is 00:00:00.028.

```
SQL> CREATE TABLE Employees (
  EmployeeID NUMBER PRIMARY KEY,
  EmployeeName VARCHAR2(50),
  DepartmentID NUMBER,
  Salary NUMBER
);
```

Table EMPLOYEES created.

Elapsed: 00:00:00.028

Cognizant Digital Nurture Program

Week 1 Hands-on

2. Insert Employee Data

The screenshot shows the SQL Worksheet interface with the following SQL script:

```
1 INSERT INTO Employees VALUES (1,'John',10,50000);
2 INSERT INTO Employees VALUES (2,'Mary',10,60000);
3 INSERT INTO Employees VALUES (3,'David',20,55000);
4
5 COMMIT;
```

The Query result tab shows the following output:

```
1 row inserted.
Elapsed: 00:00:00.002

SQL> COMMIT

Commit complete.
Elapsed: 00:00:00.002
```

3. Verify Employee Data

The screenshot shows the SQL Worksheet interface with the following SQL script:

```
1 SELECT * FROM Employees;
```

The Query result tab shows the following output:

```
Download Execution time: 0.011 seconds
```

	EMPLOYEEID	EMPLOYEENAME	DEPARTMENTID	SALARY
1	1	John	10	50000
2	2	Mary	10	60000
3	3	David	20	55000

4. Create Bonus Procedure

The screenshot shows the SQL Worksheet interface with the following SQL script:

```
1 CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (
2   p_DeptamentID NUMBER,
3   p_BonusPercent NUMBER
4 )
5 IS
6 BEGIN
7   UPDATE Employees
8   SET Salary = Salary + (Salary * p_BonusPercent/100)
9   WHERE DepartmentID = p_DeptamentID;
10
11 COMMIT;
12 END;
13 /
```

The Query result tab shows the following output:

```
SQL> CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
  p_DeptamentID NUMBER,
  p_BonusPercent NUMBER
)...
Show more...

Procedure UPDATEEMPLOYEEBONUS compiled
Elapsed: 00:00:00.031
```

Cognizant Digital Nurture Program

Week 1 Hands-on

5. Execute Procedure

The screenshot shows the SQL Developer interface. On the left is the Navigator pane with 'My Schema' selected, showing a tree of tables: ACCOUNTS, CUSTOMERS, EMPLOYEES, and LOANS. The main editor displays a PL/SQL block in a worksheet:

```
1 BEGIN
2   UpdateEmployeeBonus(10,10);
3 END;
```

Below the editor, the 'Script output' tab is active, showing the execution results:

```
Elapsed: 00:00:00.031

SQL> BEGIN
      UpdateEmployeeBonus(10,10);
      END;

PL/SQL procedure successfully completed.
Elapsed: 00:00:00.012
```

6. Verify Updated Salaries

The screenshot shows the SQL Developer interface with the same Navigator pane. The main editor displays a query:

```
1 SELECT * FROM Employees;
```

Below the editor, the 'Query result' tab is active, showing the query results in a table. The execution time is 0.016 seconds.

	EMPLOYEEID	EMPLOYEENAME	DEPARTMENTID	SALARY
1	1	John	10	55000
2	2	Mary	10	66000
3	3	David	20	55000

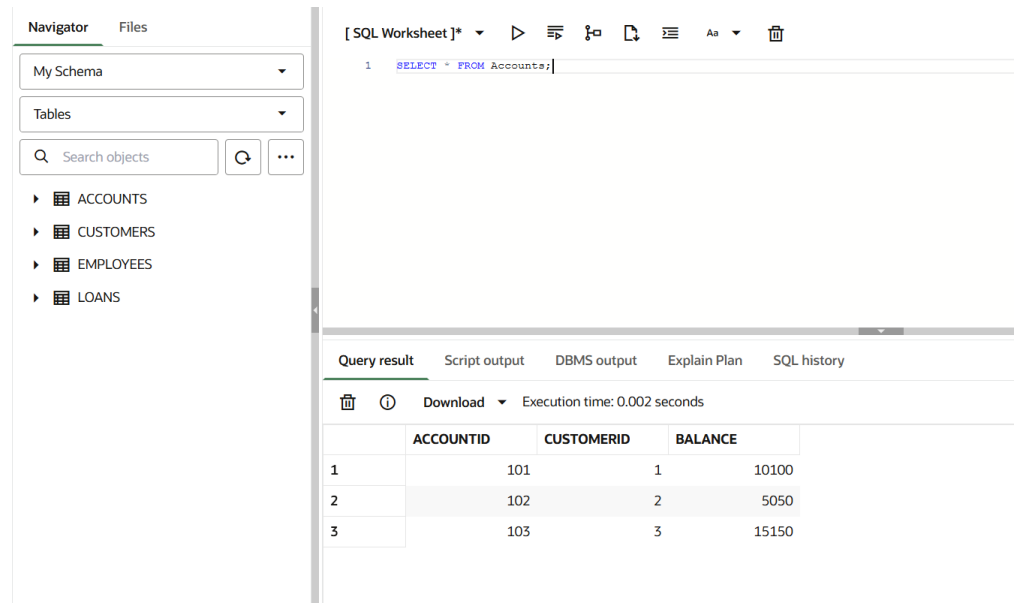
Scenario 3: Customers should be able to transfer funds between their accounts.

Question: Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

Cognizant Digital Nurture Program

Week 1 Hands-on

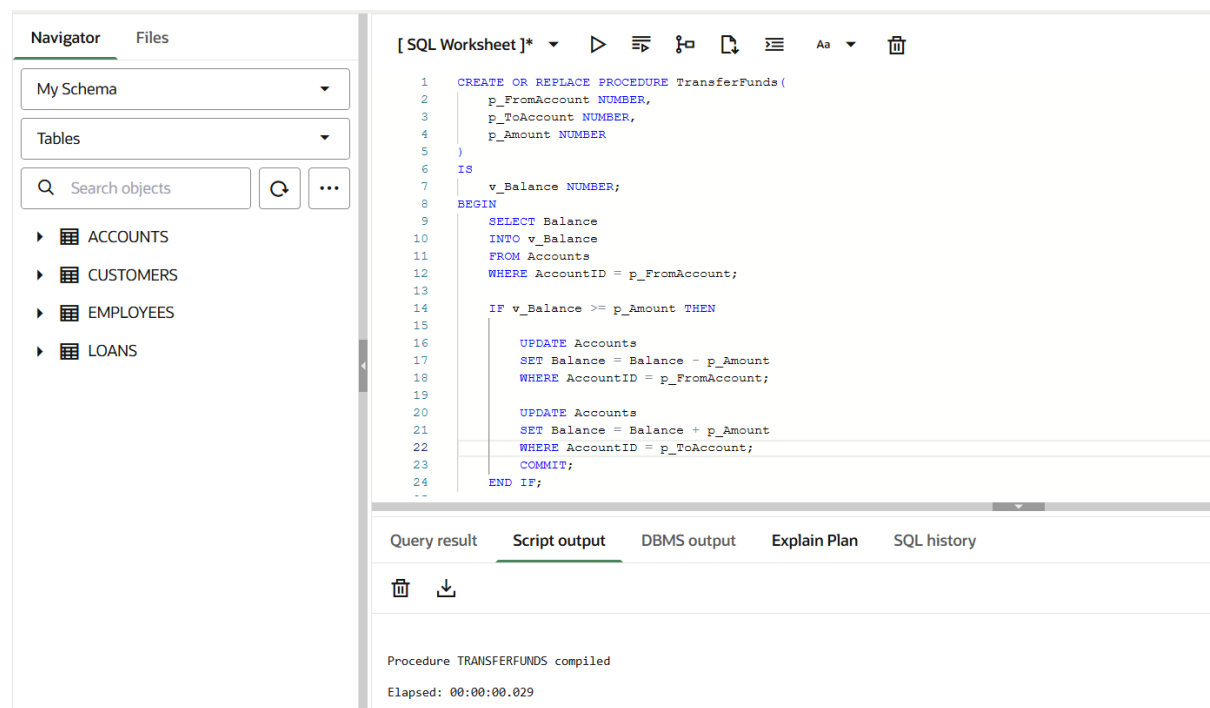
1. Check Accounts Before Transfer



The screenshot shows the SQL Developer interface. On the left, the Navigator pane displays the 'My Schema' containing tables: ACCOUNTS, CUSTOMERS, EMPLOYEES, and LOANS. The main editor shows a SQL query: `SELECT * FROM Accounts;`. Below the editor, the 'Query result' tab is active, displaying the execution time as 0.002 seconds. The result is a table with four columns: ACCOUNTID, CUSTOMERID, and BALANCE.

	ACCOUNTID	CUSTOMERID	BALANCE
1	101	1	10100
2	102	2	5050
3	103	3	15150

2. Create Transfer Procedure



The screenshot shows the SQL Developer interface. The Navigator pane on the left is the same as in the first screenshot. The main editor shows a SQL script to create a procedure named 'TransferFunds'. The script includes parameters for p_FromAccount, p_ToAccount, and p_Amount. It checks if the balance in the source account is greater than or equal to the transfer amount. If so, it updates the source account balance and the destination account balance. The script ends with a COMMIT and END IF statement.

```
1 CREATE OR REPLACE PROCEDURE TransferFunds(  
2   p_FromAccount NUMBER,  
3   p_ToAccount NUMBER,  
4   p_Amount NUMBER  
5 )  
6 IS  
7   v_Balance NUMBER;  
8 BEGIN  
9   SELECT Balance  
10  INTO v_Balance  
11  FROM Accounts  
12  WHERE AccountID = p_FromAccount;  
13  
14  IF v_Balance >= p_Amount THEN  
15  
16    UPDATE Accounts  
17    SET Balance = Balance - p_Amount  
18    WHERE AccountID = p_FromAccount;  
19  
20    UPDATE Accounts  
21    SET Balance = Balance + p_Amount  
22    WHERE AccountID = p_ToAccount;  
23    COMMIT;  
24  END IF;  
25
```

Below the editor, the 'Script output' tab is active, showing the message: 'Procedure TRANSFERFUNDS compiled' and 'Elapsed: 00:00:00.029'.

Cognizant Digital Nurture Program

Week 1 Hands-on

3. Execute Transfer

The screenshot shows the SQL Developer interface. On the left, the Navigator pane displays 'My Schema' and 'Tables'. The main editor shows a PL/SQL procedure named 'TransferFunds' with parameters (101, 102, 1000). The procedure body is as follows:

```
1 BEGIN
2   TransferFunds(101,102,1000);
3 END;
4 /
```

The 'Script output' tab is selected, showing the execution results:

```
SQL> BEGIN
      TransferFunds(101,102,1000);
    END;

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.031
```

4. Verify Transfer

The screenshot shows the SQL Developer interface. On the left, the Navigator pane displays 'My Schema' and 'Tables'. The main editor shows a SQL query to select all records from the 'Accounts' table:

```
1 SELECT * FROM Accounts;
```

The 'Query result' tab is selected, showing the execution results. The execution time is 0.003 seconds. The results are displayed in a table:

	ACCOUNTID	CUSTOMERID	BALANCE
1	101	1	9100
2	102	2	6050
3	103	3	15150

Conclusion

The PL/SQL stored procedures were successfully implemented.

Scenario 1:

Processed monthly interest for savings accounts by applying a 1% interest rate to the account balance.

Scenario 2:

Updated employee salaries by adding a bonus percentage for employees in a specified department.

Scenario 3:

Transferred funds between accounts after verifying that the source account had sufficient balance.

Thus, all stored procedure operations were executed and verified successfully.